TeleToAuto_ADY — API, Topic, Message Rate & Data-Range Reference

Source reviewed: `TeleToAuto_ADY-main.zip` **Purpose:** Map the interfaces used by each major folder, identify the source and receiving files, record rates found in the code, and distinguish configured values from recommended operating targets.

> ***Important reading rule***
>
> *- **Implemented/configured** means explicitly present in this ZIP.*
>
> *- **Recommended target** is an engineering target for operation/testing, not a value enforced by the current code.*
>
> *- Where the code does not define a numeric limit or rate, this document says **not specified** rather than inventing one.*

---

1. System overview

```
Operator browser
  webcam -> MediaPipe Hands -> dual-hand IK
  app.js sends joint_targets at 20 Hz
       |
       | WebRTC DataChannel: joint-control
       v
robot/robot1.py
  validates degrees + timestamp
  converts arm joint angles to radians
  UDP 127.0.0.1:8768
       |
       v
static/arm_bridge.py
  rate-limits and publishes Float64MultiArray at 50 Hz
       |
       +--> /right_arm_position_controller/commands
       +--> /left_arm_position_controller/commands


Operator phone (/mobile)
  DeviceOrientation -> calibrated yaw/roll
  WebSocket /ws, message head_orientation, about 30 Hz
       |
       v
server/websocket.py -> robot WebSocket
       |
       v
robot/robot1.py -> UDP 127.0.0.1:8769
       |
       v
static/gyro/gyro_to_gazebo.py
       +--> /camera_yaw_position_controller/commands
       +--> /camera_pitch_position_controller/commands


Gazebo camera
  /camera/sim_camera/image_raw
       |
       v
static/gyro/gazebo_camera_bridge.py
  JPEG + 4-byte length prefix over TCP 127.0.0.1:8767
       |
       v
robot/robot1.py -> WebRTC video track -> browser
```

The server handles login, authorization, robot presence, session ownership and WebRTC signaling. After WebRTC is established, the browser's arm-control DataChannel is peer-to-peer with `robot1.py`; it is not routed through the FastAPI server for every joint packet.

---

2. Folder-by-folder interface map

2.1 `client/` — browser operator interface

Interface / data	Direction	Producer file	Consumer file	Implemented rate	Data format / range
Operator webcam frames	Camera → browser processing	Browser `getUserMedia()` in `hand-tracker.js`	`hand-tracker.js` / MediaPipe Hands	Requests 30 FPS, max 30; actual camera FPS may be lower	Requests 640×480; RGB/BGR conversion is handled by MediaPipe/browser
Hand landmarks	`hand-tracker.js` → IK callback	`hand-tracker.js`	`app.js` callback → `dual-hand-ik-controller.js`	Frame-driven; no fixed processing Hz	MediaPipe Hands landmarks: 21 points per detected hand; normalized image coordinates (x/y typically 0–1; z is relative depth). Handedness and quality metadata accompany detections
IK joint target state	IK → app	`dual-hand-ik-controller.js`	`app.js`	Updated when hand observations are processed	Two sides (`left`, `right`), five named arm joints each; angles are degrees when sent on the network. Gripper state is boolean closed/open
`joint_targets` packet	Browser → robot	`client/app.js` via `robot-connection.js`	`robot/robot1.py` on `joint-control`	**20 Hz** (`setInterval(..., 50 ms)`)	JSON schema below. Timestamp is epoch ms; arm joint values in degrees
`head_orientation` packet	Phone page → robot (via server WebSocket)	`client/mobile-gyro.js`	`server/websocket.py` → `robot/robot1.py`	**~30 Hz** (`SEND_INTERVAL_MS=33`)	yaw/pitch/roll in radians. Current mobile UI clamps yaw to ±80° and roll to ±60°; pitch is sent as 0
Mirrored phone readout	Server → laptop browser	`server/websocket.py`	`client/app.js`	Same incoming rate as phone, approximately 30 Hz	`{type:"mobile_imu_readout", gx, gy, units:"deg", timestamp_ms}`. In this code `gx` is yaw and `gy` is pitch in the readout, despite robot camera vertical movement using roll
Robot video track	Robot → browser	`robot/robot1.py`	`client/robot-connection.js` → `ui-controller.js`	Target camera rate is 30 FPS; actual depends on Gazebo sensor and transport	WebRTC video track; source is Gazebo simulated camera
Calibration REST data	Server → browser	`server/routes.py`	Client calibration consumer (if used)	On request; no streaming rate	JSON calibration object

`joint_targets` JSON packet

Produced by `client/robot-connection.js`:

```
{
  "type": "joint_targets",
  "sequence": 123,
  "timestamp_ms": 1730000000000,
  "units": "deg",
  "source": "browser_hand_tracking",
  "arms": {
    "left": {
```

```
        "shoulder_pan": 0.0,
        "shoulder_lift": 0.0,
        "elbow_flex": 0.0,
        "wrist_flex": 0.0,
        "wrist_roll": 0.0
      },
      "right": {
        "shoulder_pan": 0.0,
        "shoulder_lift": 0.0,
        "elbow_flex": 0.0,
        "wrist_flex": 0.0,
        "wrist_roll": 0.0
      }
    },
    "gripper": {
      "left_closed": false,
      "right_closed": false
    }
  }
```

• Five arm joint angles per arm, plus one gripper boolean per arm.

• `robot1.py` rejects non-finite values, timestamps more than 1,000 ms away from its clock, and arm values with absolute magnitude above 180°.

• `arm_bridge.py` then clamps the converted radians to the URDF limits documented in Section 4.

• **Do not treat ±180° as the actual mechanical travel range**; it is only the broad packet validation bound.

`head_orientation` JSON packet

```
  {
    "type": "head_orientation",
    "sequence": 123,
    "timestamp_ms": 1730000000000,
    "units": "rad",
    "orientation": {
      "yaw": 0.0,
      "pitch": 0.0,
      "roll": 0.0
    },
    "source": "phone_yaw_roll_old_axis_mapping_smoothed"
  }
```

• Phone UI generates approximately 30 packets/s.

• Yaw is clamped to ±80° (±1.396 rad); roll to ±60° (±1.047 rad); pitch is set to zero by this mobile code.

• `robot1.py` validates each orientation component to ±π and timestamp age within 1,000 ms.

• `gyro_to_gazebo.py` applies a narrower output clamp: yaw ±1.4 rad, roll ±1.0 rad.

Client module responsibilities

File	Interface responsibility
`app.js`	Starts tracking, passes hand data to IK, sends arm targets every 50 ms, handles stale tracking
`hand-tracker.js`	Webcam acquisition, MediaPipe Hands, 21-point landmarks, handedness/quality
`dual-hand-ik-controller.js`	Converts hand features into left/right arm joint targets and gripper states
`robot-connection.js`	WebSocket signaling setup, WebRTC peer connection, DataChannels, packet serialization
`mobile-gyro.js`	Phone DeviceOrientation, calibration/filtering, yaw/roll packet generation
`ui-controller.js`	UI and video attachment; not a robot command transport
`digital-twin.js`	Browser-side visualization; no robot-control ROS topic is declared here
`manipulator-controller.js`	Imported by the current app as the manipulator controller facade; verify its internal use when modifying IK integration

---

2.2 `server/` — REST API and WebSocket signaling

HTTP REST endpoints

Method + path	Purpose	Source file	Request / response	Rate
`GET /`	Serve operator page	`server/routes.py`	HTML	On page load
`GET /mobile`	Serve phone orientation page	`server/routes.py`	HTML	On page load
`GET /api/health`	Liveness + online robot IDs	`server/routes.py`	Returns `{ok, online_robots}`	Polling/on demand; no polling interval set in server
`POST /api/login`	Authenticate user	`server/routes.py`, `server/auth.py`	`{username,password}` → `{success,token,username}` or error	On login
`GET /api/robots`	List robots visible to authenticated user	`server/routes.py`, `server/robot_manager.py`	Returns `{robots:[...]}`	On demand
`GET /api/calibration?id=...`	Return matching calibration file or fallback	`server/routes.py`, `server/config.py`	Returns `{success,source,calibration}`	On demand

`server/routes.py` also serves the JavaScript modules as static GET routes. The favicon and Chrome DevTools discovery routes are browser support routes, not robot-control APIs.

WebSocket endpoint: `ws://<host>:<port>/ws` or `wss://.../ws`

Implemented in `server/websocket.py`.

First message identifies and authenticates the role:

Role	First message fields	Authentication source	Purpose
`robot`	`role`, `robot_id`, `token`	`server/auth.py` / robot credentials	Registers robot presence and receives signaling/control-forward messages
`browser`	`role`, `robot_id`, `token`	User session token + robot permission	Reserves the robot for one browser operator
`mobile_gyro`	`role`, `robot_id`, `token`	User session token + robot permission	Separate phone session; does not replace browser slot

Signaling message types include:

- `authenticated`, `connected`, `mobile_gyro_connected`, `error`
- `browser_ready`, `mobile_ready`
- `offer`, `answer`, `candidate`
- `mobile_disconnected`, `browser_disconnected`
- `head_orientation` from phone, forwarded to robot and mirrored to browser as `mobile_imu_readout`

WebSocket signaling is event-driven. The server does **not** specify a periodic heartbeat/data rate for the application messages. WebSocket library ping settings for the robot connection are separate from control packet rates.

Server file responsibilities

File	Responsibility
`main.py`	FastAPI app assembly and router inclusion
`routes.py`	HTTP routes and static client file serving
`websocket.py`	`/ws`, role authentication, signaling and phone orientation forwarding
`robot_manager.py`	Online robot registry, browser/mobile ownership and message routing

File	Responsibility
`auth.py`	Login/session and access checks
`config.py`	Paths, calibration lookup and configuration

2.3 `robot/` — robot-side WebRTC, validation and local transport

Active file: `robot/robot1.py`

Interface	Direction	Producer	Consumer	Rate	Data / constraints
WebSocket `/ws`	Robot ↔ server	`robot1.py`	`server/websocket.py`	Event-driven	Robot authentication, ready messages, SDP offer/answer, ICE, mobile orientation forwarding
WebRTC `joint-control`	Browser → robot	`client/robot-connection.js`	`robot1.py`	Browser sends 20 Hz	`joint_targets`, degrees, five named joints/arm + boolean grippers
WebRTC `imu-control`	Browser → robot	Browser channel is created, but current phone orientation path uses server WS	`robot1.py`	No current browser-side send rate established in this ZIP	Channel is created by robot for browser session; `client/robot-connection.js` supports `head_orientation`, but `app.js` does not send it from laptop
UDP `127.0.0.1:8768`	`robot1.py` → arm bridge	`robot1.py`	`static/arm_bridge.py`	Follows incoming joint packets, normally 20 Hz	JSON `joint_targets`; adds `arms_rad`, `gripper_rad`, `received_at`
UDP `127.0.0.1:8769`	`robot1.py` → gyro bridge	`robot1.py`	`static/gyro/gyro_to_gazebo.py`	Follows phone orientation, normally ~30 Hz	JSON `head_orientation`; yaw/roll radians
TCP `127.0.0.1:8767`	Camera bridge → robot receiver	`static/gyro/gazebo_camera_bridge.py`	`robot1.py` frame receiver	Source camera rate; target 30 FPS	4-byte network-order JPEG length followed by JPEG bytes
WebRTC video track	Robot → browser/phone viewer	`robot1.py`	`client/robot-connection.js`	Target 30 FPS; actual is measured/variable	VideoFrame from latest Gazebo camera image

Important: `robot1.py` has `FOLLOWER_ENABLED = False`; this ZIP's active robot1 path is simulation/Gazebo-oriented, not physical SO-101 follower actuation.

`robot1.py` safety validation

• Joint packet type must be `joint_targets`.

• Joint timestamp must be within ±1,000 ms of robot clock.

• All five named joint values on both sides must be finite and `abs(value) <= 180°`.

• Gripper fields are normalized to booleans.

• Phone orientation values must be finite and within ±π; timestamp within ±1,000 ms.

• Gyroscope rate values, if `head_gyro` packets are used, must be finite and within ±2,000 deg/s.

• `COMMAND_TIMEOUT_SECONDS` is configured as 0.5 s in `robot1.py`; inspect the watchdog/timeout handling before relying on it as an actuator-stop guarantee.

`robot2.py` status

`robot2.py` is a separate/older robot process and uses a different protocol:

• WebRTC DataChannel: `encoder-control`

• Message type: `joint_positions`

• Payload includes `positions` keys such as `shoulder_pan.pos`, `shoulder_lift.pos`, `elbow_flex.pos`, `wrist_flex.pos`, `wrist_roll.pos`, `gripper.pos`.

• Its code logs received rate but does not define a required command rate in the inspected section.

• This is **not the same packet schema** as active `robot1.py`'s `joint-control` / `joint_targets` path. Do not mix the two without an explicit protocol adapter.

2.4 `static/` — ROS 2 / Gazebo bridge interfaces

Arm bridge: `static/arm_bridge.py`

ROS topic	Direction	Message type	Publisher file	Subscriber / controller	Rate	Data
`/right_arm_position_controller/commands`	Bridge → ROS controller	`std_msgs/msg/Float64MultiArray`	`arm_bridge.py`	`right_arm_position_controller` in `humanoid_controllers.yaml`	**50 Hz** configured	`data` length 6: 5 arm joints + gripper, radians
`/left_arm_position_controller/commands`	Bridge → ROS controller	`std_msgs/msg/Float64MultiArray`	`arm_bridge.py`	`left_arm_position_controller` in `humanoid_controllers.yaml`	**50 Hz** configured	`data` length 6: 5 arm joints + gripper, radians

Additional arm bridge details:

• UDP listener: `127.0.0.1:8768`.

• UDP polling timer: 5 ms (200 Hz poll loop), but this is not the ROS publish rate.

• ROS publish timer: 50 Hz.

• Packet timeout parameter: 1.0 s.

• Max arm joint speed: 3.0 rad/s.

• Max gripper speed: 4.0 rad/s.

• The bridge keeps only the latest received packet and ramps current joint values toward target.

• Gripper commands: right open `-1.2 rad`, closed `0`; left open `-1.5 rad`, closed `0` (parameters).

• These are bounded by the joint limits below.

Gyro bridge: `static/gyro/gyro_to_gazebo.py`

ROS topic	Direction	Message type	Publisher file	Controller	Rate	Data
`/camera_yaw_position_controller/commands`	Bridge → ROS controller	`std_msgs/msg/Float64MultiArray`	`gyro_to_gazebo.py`	`camera_yaw_position_controller`	On each received orientation packet; phone target ~30 Hz	`data` length 1, yaw radians
`/camera_pitch_position_controller/commands`	Bridge → ROS controller	`std_msgs/msg/Float64MultiArray`	`gyro_to_gazebo.py`	`camera_pitch_position_controller`	On each received orientation packet; phone target ~30 Hz	`data` length 1, phone roll radians

Additional gyro bridge details:

• UDP listener: `0.0.0.0:8769`.

• UDP polling timer: 10 ms (100 Hz); this is not the incoming packet rate.

• Yaw output clamp: `[-1.4, +1.4] rad`.

• Roll output clamp: `[-1.0, +1.0] rad`.

• `head_gyro` mode integrates gx/gy in deg/s over elapsed time, if such packets are supplied.

• The current phone page sends absolute `head_orientation`, not `head_gyro`.

Gazebo camera bridge: `static/gyro/gazebo_camera_bridge.py`

ROS topic	Direction	Message type	Subscriber file	Rate	Data
`/camera/sim_camera/image_raw`	Gazebo → bridge	`sensor_msgs/msg/Image`	`gazebo_camera_bridge.py`	Gazebo sensor rate; inspect URDF sensor block for configured rate	Image converted to BGR then JPEG encoded at quality 55

Transport:

• TCP server/listener is in `robot1.py` at `127.0.0.1:8767`.

• `gazebo_camera_bridge.py` connects as TCP client.

• Each frame is sent as a 4-byte big-endian length followed by JPEG payload.

• The bridge holds only the latest JPEG frame; old frames are overwritten rather than queued.

Controller manager

`static/humanoid_controllers.yaml`:

• Controller manager `update_rate: 100` Hz.

• `use_sim_time: true`.

• Controllers: joint state broadcaster, camera yaw, camera pitch, right arm, left arm.

• This 100 Hz is the controller-manager update rate, not the rate at which `arm_bridge.py` publishes arm commands.

Launch files

• `static/gazebo_humanoid.launch.py`: launches Gazebo, robot state publisher, spawns robot, then starts controller spawners.

• `static/gyro/camera_gyro_bridges.launch.py`: launches camera bridge and gyro bridge.

• Launch files orchestrate nodes; they do not define the control packet schemas.

2.5 `static/so101_follower/` — alternate SO-101 simulation branch

This folder contains a separate SO-101 dual-arm Gazebo/IK implementation:

• `dual_hand_ik_teleop.py`

• `so101_dual_gazebo.launch.py`

• `so101_new_calib.urdf`

It should be treated as a separate simulation/control path from the humanoid path in `static/arm_bridge.py` unless launch/configuration explicitly connects them. Do not assume its topics or rates are the same as the humanoid topics listed above. Inspect its launch file and node code before integrating.

2.6 `Others/` — legacy physical LeRobot files

Contains:

- `follower_controller.py`
- `follower_driver.py`
- `follower_before.json`
- `leader_before.json`
- `arduino_code.cpp`

These are legacy/physical leader-follower or microcontroller artifacts. They are not in the active `robot1.py` simulation data path described above. No active ROS topic or API from these files should be assumed to be connected to the current browser → robot1 → Gazebo flow.

---

3. Rate summary — configured versus recommended

Interface	Current code rate	Suggested operating target	Notes
Webcam capture for hand tracking	Requests up to 30 FPS	25–30 FPS sustained	Browser/device dependent
MediaPipe hand processing	Frame-driven, no fixed cap	20–30 processed FPS	Current loop uses `requestAnimationFrame` and one-frame-at-a-time guard
Browser arm target packets	**20 Hz**	20–30 Hz	Current `app.js` timer is 50 ms; increase only after latency/CPU testing
Phone orientation packets	**~30 Hz**	30 Hz	Current 33 ms interval
Robot-to-UDP arm forwarding	Incoming packet-driven (~20 Hz)	Match browser rate	UDP is latest-value style
Arm bridge ROS publishing	**50 Hz**	50–100 Hz if simulation/controller performance supports it	Current configured 50 Hz
Gyro bridge ROS publishing	Incoming-packet-driven (~30 Hz)	Match phone rate, ~30 Hz	10 ms timer is only socket polling
Controller manager	**100 Hz**	100 Hz	Configured in YAML
Gazebo camera image	Not confirmed from bridge file	30 FPS if URDF sensor supports it	Inspect camera sensor `<update_rate>`
Camera TCP/WebRTC video	Source-frame-driven; target 30 FPS	20–30 FPS	Actual throughput depends on Gazebo, JPEG encode, network and browser
Server REST	On demand	On demand	No polling interval imposed in route code
Server signaling WebSocket	Event-driven	Event-driven	Not a periodic control channel

---

4. Joint and control data ranges

4.1 Arm joint limits used by `static/arm_bridge.py`

These are the bridge's configured URDF limits in radians, ordered:

```
[shoulder_pan, shoulder_lift, elbow_flex, wrist_flex, wrist_roll, gripper]
```

Arm	Joint	Minimum (rad)	Maximum (rad)
Right	shoulder_pan	-1.91986	+1.91986

Arm	Joint	Minimum (rad)	Maximum (rad)
Right	shoulder_lift	-1.74533	+1.74533
Right	elbow_flex	-1.69000	+1.69000
Right	wrist_flex	-1.65806	+1.65806
Right	wrist_roll	-2.74385	+2.84121
Right	gripper	-1.34300	+0.13700
Left	shoulder_pan	-1.91986	+1.91986
Left	shoulder_lift	-1.74533	+1.74533
Left	elbow_flex	-1.69000	+1.69000
Left	wrist_flex	-1.65806	+1.65806
Left	wrist_roll	-2.74385	+2.84121
Left	gripper	-1.70000	+0.22000

The bridge clamps all six values to these limits before publishing. Gripper open/closed parameter values are set separately and then clamped.

4.2 IK workspace and hand tracking thresholds

Values from `client/dual-hand-ik-controller.js`:

Parameter	Configured value	Meaning
Home XYZ	`[0.28, 0.012, -0.24]` m	Neutral tool position relative to shoulder-pan frame
Workspace range	`[0.09, 0.20, 0.09]` m	X/Y/Z half-range scaling used by IK
Hand span defaults	X 0.22 m, Y 0.20 m	Uncalibrated hand mapping spans
Depth span	0.35	Normalized hand-size/depth mapping parameter
Minimum hand quality	0.25	Below this, observation is ignored
Minimum pinch quality	0.5	Below this, gripper retains previous state
Lost-hand hold	500 ms	Holds last valid pose for this interval
Stale target timeout	500 ms	`app.js` pauses sending stale targets
Max IK target speed	1.5 m/s	Target movement limiter
Reacquire target speed	0.2 m/s	Initial speed after reacquiring a hand
Reacquire ramp	800 ms	Ramp back toward normal target speed
Max joint speed in IK	3.0 rad/s	IK joint change limiter
Gripper pinch thresholds	closed 0.25, open 0.95	Normalized pinch feature thresholds

The exact output joint angles are dynamic IK results, not a fixed operating range. The bridge's joint limits are the final clamp.

4.3 Phone orientation

Axis	Mobile page input clamp	Robot bridge output clamp	Unit
Yaw	±80°	±1.4 rad (about ±80.2°)	Radians in transmitted packet and ROS command
Roll	±60°	±1.0 rad (about ±57.3°)	Radians
Pitch	Sent as 0 by current phone page	Not used by `gyro_to_gazebo.py` for camera command	Radians

Note: `server/websocket.py` mirrors `gx` from yaw and `gy` from pitch to the browser readout. The Gazebo vertical camera command uses **roll**. This naming difference matters when debugging the UI display.

---

5. ROS topic quick reference

Topic	Type	Publisher	Expected data length	Rate
`/right_arm_position_controller/commands`	`std_msgs/msg/Float64MultiArray`	`static/arm_bridge.py`	6 doubles, radians	50 Hz
`/left_arm_position_controller/commands`	`std_msgs/msg/Float64MultiArray`	`static/arm_bridge.py`	6 doubles, radians	50 Hz
`/camera_yaw_position_controller/commands`	`std_msgs/msg/Float64MultiArray`	`static/gyro/gyro_to_gazebo.py`	1 double, radians	~30 Hz when phone active
`/camera_pitch_position_controller/commands`	`std_msgs/msg/Float64MultiArray`	`static/gyro/gyro_to_gazebo.py`	1 double, radians	~30 Hz when phone active
`/camera/sim_camera/image_raw`	`sensor_msgs/msg/Image`	Gazebo camera sensor	Image message	URDF sensor update rate; not confirmed here

The joint-state broadcaster is configured, but the exact joint-state topic is the standard controller output and is not used as an input command topic in this control path.

---

6. Interfaces not present or not confirmed in this ZIP

These are important so that future integration does not accidentally document planned features as implemented:

• **AMR** `cmd_vel`**:** no active AMR velocity topic publisher/subscriber was found in the reviewed active browser/robot/server files.

• **ESP32 joystick/buttons/IMU:** the current `app.js` explicitly reports that a standalone ESP32/Bluetooth IMU module is not included. Phone orientation is implemented separately at `/mobile`.

• **Dedicated physical second camera:** the active `robot1.py` path is configured to use the Gazebo simulated camera bridge. A physical camera is disabled by default.

• **Laptop/browser IMU sender:** `robot-connection.js` has `sendHeadOrientation()`, but the current `app.js` does not call it; the phone path is the implemented orientation sender.

• **Fixed MediaPipe processing rate:** the tracker uses `requestAnimationFrame`; no explicit fixed Hz throttle is configured.

• **Camera sensor update rate and resolution:** the bridge topic is known, but the authoritative `<update_rate>`, width and height must be read from the camera sensor block in `static/urdf/humanoid_gazebo.urdf`.

• `robot2.py` **compatibility:** its `encoder-control` / `joint_positions` protocol is different from `robot1.py`'s `joint-control` / `joint_targets` protocol.

---

7. Verification commands

Run these while the simulation is active.

Check ROS topics and message types

```
ros2 topic list -t
```

Check arm command rate

```
ros2 topic hz /right_arm_position_controller/commands
ros2 topic hz /left_arm_position_controller/commands
```

Expected from current bridge configuration: approximately 50 Hz after targets have arrived.

Check camera orientation command rate

```
ros2 topic hz /camera_yaw_position_controller/commands
ros2 topic hz /camera_pitch_position_controller/commands
```

Expected while the phone is actively sending: approximately 30 Hz.

Inspect topic values

```
ros2 topic echo /right_arm_position_controller/commands
ros2 topic echo /left_arm_position_controller/commands
ros2 topic echo /camera_yaw_position_controller/commands
ros2 topic echo /camera_pitch_position_controller/commands
```

Check camera image stream

```
ros2 topic info /camera/sim_camera/image_raw
ros2 topic hz /camera/sim_camera/image_raw
```

Check WebSocket health API

```
curl http://127.0.0.1:8000/api/health
```

Check open local ports

```
ss -lntup | grep -E '5000|8000|8767|8768|8769'
```

---

8. Source files inspected

- `client/app.js`
- `client/hand-tracker.js`
- `client/dual-hand-ik-controller.js`
- `client/robot-connection.js`
- `client/mobile-gyro.js`
- `server/main.py`
- `server/routes.py`
- `server/websocket.py`
- `server/robot_manager.py`
- `server/auth.py`
- `server/config.py`
- `robot/robot1.py`
- `robot/robot2.py`
- `static/arm_bridge.py`
- `static/gyro/gyro_to_gazebo.py`
- `static/gyro/gazebo_camera_bridge.py`
- `static/humanoid_controllers.yaml`
- `static/urdf/humanoid_gazebo.urdf`
- `static/gazebo_humanoid.launch.py`
- `static/gyro/camera_gyro_bridges.launch.py`
- `static/so101_follower/dual_hand_ik_teleop.py`
- `static/so101_follower/so101_dual_gazebo.launch.py`
- `README.md`, `SETUP.md`, `NOTES.md`

Documenting note: The values above describe the code in the supplied ZIP. They are not a claim that all interfaces are currently running simultaneously. Use the verification commands to measure actual runtime rates on the target machine.